Assignment 9: Pattern Matching and Environment Semantics

CAS CS 320: Concepts of Programming Languages

due Thursday, November 19 by 8:00PM

For the programming tasks, you'll put your code in the file `lib/assign9.ml`. You'll submit it via Gradescope, where your code will be checked against a collection of hidden tests. You must pass **3/4** of the Gradescope unit tests on each problem for the assignment to be considered attempted. Your code must also follow the guidelines of the course for it to be considered attempted (i.e., it may not use any references, arrays, mutable fields, or for/while loops).

You're not required to submit any written problems this week.

You're required to submit solutions to every problem in this assignment to receive full credit, but we will only provide detailed feedback on **bolded** problems. Since the assignment is graded on participation, we ask that you *try at least the bolded problems with as few external aids as possible.* You should think of these problems as simulated quizzes or exams. *You won't be penalized for mistakes*, so it's in your best interest to do these problems on your own; the detailed feedback we provide will generally be more useful (for you and for us) if you submit your own work.

Programming

Outline. This week you'll be filling in the details of an interpreter for a PL with recursion, lists and (deep) pattern matching. You should think of this as a "warm-up" for the final project. You'll need to fill in parts of the following functions.

```
val type_pat : Pattern.t -> Type.t -> (Ctxt.t, Ty_error.t) result
val type_of : Ctxt.t -> Expr.t -> (Type.t, Ty_error.t) result
val matches : Value.t -> Pattern.t -> Env.t option (* bolded *)
val eval : Env.t -> Expr.t -> Value.t
```

Syntax

We'll be using S-expressions for the concrete syntax.

```
<expr> ::= ( let <var> <ty> <expr> <expr> )
         | ( if <expr> <expr> <expr> )
         | ( lambda <var> <ty> <expr> )
         | ( <expr> <expr> )
         | ( <op> <expr> <expr> )
         | ( match <expr> <arms> )
         | ( [] <ty> )
         | true | false | <var> | <int>
  <ty> ::= int | bool | ( -> <ty> <ty> ) | ( list <ty> )
  <op> ::= + | - | * | = | <> | < | <= | > | >=
<arms> ::= ( <arm> {<arm>} )
 <arm> ::= ( <pat> <expr> )
 <pat> ::= ( :: <pat> <pat> ) | <var> | <int>
         | true | false | [] | _
```

where <int> is a nonempty sequence of digits, optionally preceded by a minus sign (the same strings recognized by `int_of_string_opt`), <var> is a nonempty sequence of lowercase English letters or underscores, not starting with an underscore (so as to distinguish from the wildcard _ There's nothing to implement here; we've given you the parser.

Typing

The typing rules are given in terms of the following abstract syntax:

$$
\begin{array}{rcll}
e & ::= & \lambda x^{\tau}.e \mid ee \mid \mathsf{let}\ x^{\tau} = e\ \mathsf{in}\ e \mid \mathsf{if}\ e\ \mathsf{then}\ e\ \mathsf{else}\ e \mid e\ o\ e & \text{(expressions)} \\
 & \mid & \mathsf{match}\ e\ \mathsf{with}\ a\ \{a\} \mid [\,]^{\tau} \mid \mathsf{true} \mid \mathsf{false} \mid x \mid n & \\
o & ::= & + \mid - \mid \times \mid = \mid \neq \mid < \mid \leq \mid > \mid \geq \mid :: & \text{(operators)} \\
\tau & ::= & \tau \to \tau \mid \mathsf{int} \mid \mathsf{bool} \mid \mathsf{List}(\tau) & \text{(types)} \\
a & ::= & \restriction\ p \to e & \text{(match arms)} \\
p & ::= & \mathsf{true} \mid \mathsf{false} \mid n \mid [\,] \mid p :: p \mid x \mid _ & \text{(patterns)} \\
v & ::= & \mathsf{true} \mid \mathsf{false} \mid n \mid (\!|\ \mathcal{E}\ ,\ \lambda x.e\ |\!) \mid (\!|\ \mathcal{E}\ ,\ \lambda x.e\ ,\ x\ |\!) \mid [\,] \mid v :: v & \text{(values)} \\
x & & & \text{(variables)} \\
n & & & \text{(integers)} \\
\mathcal{E} & & & \text{(environments)}
\end{array}
$$

Typing rules:

$$\frac{(x:\tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{var} \qquad \frac{n \in \mathbb{Z}}{\Gamma \vdash n : \mathsf{int}}\ \textsc{int} \qquad \frac{}{\Gamma \vdash \mathsf{true} : \mathsf{bool}}\ \textsc{true} \qquad \frac{}{\Gamma \vdash \mathsf{false} : \mathsf{bool}}\ \textsc{false}$$

$$\frac{\Gamma \vdash e_1 : \mathsf{int} \qquad \Gamma \vdash e_2 : \mathsf{int}}{\Gamma \vdash e_1 + e_2 : \mathsf{int}}\ \textsc{add} \qquad \frac{\Gamma \vdash e_1 : \mathsf{int} \qquad \Gamma \vdash e_2 : \mathsf{int}}{\Gamma \vdash e_1 - e_2 : \mathsf{int}}\ \textsc{sub} \qquad \frac{\Gamma \vdash e_1 : \mathsf{int} \qquad \Gamma \vdash e_2 : \mathsf{int}}{\Gamma \vdash e_1 \times e_2 : \mathsf{int}}\ \textsc{mul}$$

$$\frac{\Gamma \vdash e_1 : \mathsf{int} \qquad \Gamma \vdash e_2 : \mathsf{int}}{\Gamma \vdash e_1 \leq e_2 : \mathsf{bool}}\ \textsc{leq} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x^{\tau_1}.e : \tau_1 \to \tau_2}\ \textsc{fun} \qquad \frac{\Gamma \vdash e_1 : \tau_2 \to \tau \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau}\ \textsc{app}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathsf{let}\ x^{\tau_1} = e_1\ \mathsf{in}\ e_2 : \tau_2}\ \textsc{let} \qquad \frac{\Gamma \vdash e_1 : \mathsf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau}\ \textsc{if}$$

$$\frac{}{\Gamma \vdash [\,]^{\tau} : \mathsf{List}(\tau)}\ \textsc{nil} \qquad \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \mathsf{List}(\tau)}{\Gamma \vdash e_1 :: e_2 : \mathsf{List}(\tau)}\ \textsc{cons}$$

$$\frac{\Gamma \vdash e_0 : \tau_0 \qquad p_1 : \tau_0 \Rightarrow \Gamma_1 \qquad \Gamma, \Gamma_1 \vdash e_1 : \tau \qquad \ldots \qquad p_k : \tau_0 \Rightarrow \Gamma_k \qquad \Gamma, \Gamma_k \vdash e_k : \tau}{\Gamma \vdash \mathsf{match}\ e_0\ \mathsf{with}\ \ \restriction\ p_1 \to e_1 \ldots\ \restriction\ p_k \to e_k : \tau}\ \textsc{match}$$

Pattern typing rules:

$$\frac{}{\mathsf{true} : \mathsf{bool} \Rightarrow \{\}}\ \textsc{trueP} \qquad \frac{}{\mathsf{false} : \mathsf{bool} \Rightarrow \{\}}\ \textsc{falseP} \qquad \frac{n \in \mathbb{Z}}{n : \mathsf{int} \Rightarrow \{\}}\ \textsc{intP} \qquad \frac{}{[\,] : \mathsf{List}(\tau) \Rightarrow \{\}}\ \textsc{nilP}$$

$$\frac{p_1 : \tau \Rightarrow \Gamma_1 \qquad p_2 : \mathsf{List}(\tau) \Rightarrow \Gamma_2 \qquad \Gamma_1 \text{ and } \Gamma_2 \text{ are disjoint}}{p_1 :: p_2 : \mathsf{List}(\tau) \Rightarrow \Gamma_1, \Gamma_2}\ \textsc{consP} \qquad \frac{}{x : \tau \Rightarrow \{x : \tau\}}\ \textsc{varP}$$

$$\frac{}{_ : \tau \Rightarrow \{\}}\ \textsc{wildP}$$

You might notice that the above is, strictly speaking, a subset of the required rules (in particular, there's no rule for recursive let expressions). You'll need to think about how the rules should work in the missing cases. You need to fill in the implementations of:

```
val type_pat : Pattern.t -> Type.t -> (Ctxt.t, Ty_error.t) result
val type_of : Ctxt.t -> Expr.t -> (Type.t, Ty_error.t) result
```

so that `type_of gamma e` is `Ok t` if e has type t in the context Γ, i.e., $\Gamma \vdash e : t$ is derivable according to the above rules. Otherwise, it should be `Error err` where `err` is of type `Ty_error.t` and describes the error. You'll need to look at this type (in particular, how it's stringified) to figure out what error to produce in a given case.

Semantics

You'll be implementing a big-step CBV environment semantics. The rules are defined over the same abstract syntax as above.

Semantic rules:

$$\frac{}{\langle\ \mathcal{E}\ ,\ x\ \rangle \Downarrow \mathcal{E}(x)}\ \textsc{varE} \qquad \frac{}{\langle\ \mathcal{E}\ ,\ n\ \rangle \Downarrow n}\ \textsc{intE} \qquad \frac{}{\langle\ \mathcal{E}\ ,\ \mathsf{true}\ \rangle \Downarrow \mathsf{true}}\ \textsc{trueE} \qquad \frac{}{\langle\ \mathcal{E}\ ,\ \mathsf{false}\ \rangle \Downarrow \mathsf{false}}\ \textsc{falseE}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow v_1 \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2}{\langle\ \mathcal{E}\ ,\ e_1 + e_2\ \rangle \Downarrow v_1 + v_2}\ \textsc{addE} \qquad \frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow v_1 \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2}{\langle\ \mathcal{E}\ ,\ e_1 - e_2\ \rangle \Downarrow v_1 - v_2}\ \textsc{subE}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow v_1 \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2}{\langle\ \mathcal{E}\ ,\ e_1 \times e_2\ \rangle \Downarrow v_1 \times v_2}\ \textsc{mulE} \qquad \frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow v_1 \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2 \qquad v_1 \leq v_2}{\langle\ \mathcal{E}\ ,\ e_1 \leq e_2\ \rangle \Downarrow \mathsf{true}}\ \textsc{leqTrue}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow v_1 \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2 \qquad v_1 > v_2}{\langle\ \mathcal{E}\ ,\ e_1 \leq e_2\ \rangle \Downarrow \mathsf{false}}\ \textsc{leqFalse} \qquad \frac{}{\langle\ \mathcal{E}\ ,\ \lambda x^{\tau}.e\ \rangle \Downarrow (\!|\ \mathcal{E}\ ,\ \lambda x.e\ |\!)}\ \textsc{funE}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow (\!|\ \mathcal{E}'\ ,\ \lambda x.e\ |\!) \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2 \qquad \langle\ \mathcal{E}'[x \mapsto v_2]\ ,\ e\ \rangle \Downarrow v}{\langle\ \mathcal{E}\ ,\ e_1 e_2\ \rangle \Downarrow v}\ \textsc{appE}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow (\!|\ \mathcal{E}'\ ,\ \lambda x.e\ ,\ x'\ |\!) \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2 \qquad \langle\ \mathcal{E}'[x' \mapsto (\!|\ \mathcal{E}'\ ,\ \lambda x.e\ ,\ x'\ |\!)][x \mapsto v_2]\ ,\ e\ \rangle \Downarrow v}{\langle\ \mathcal{E}\ ,\ e_1 e_2\ \rangle \Downarrow v}\ \textsc{appRecE}$$

$$\frac{e_1 \neq \lambda x^{\tau}.e \qquad \langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow v_1 \qquad \langle\ \mathcal{E}[x \mapsto v_1]\ ,\ e_2\ \rangle \Downarrow v_2}{\langle\ \mathcal{E}\ ,\ \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2\ \rangle \Downarrow v_2}\ \textsc{letE}$$

$$\frac{\langle\ \mathcal{E}[x \mapsto (\!|\ \mathcal{E}\ ,\ \lambda x_1.e_1\ ,\ x\ |\!)]\ ,\ e_2\ \rangle \Downarrow v}{\langle\ \mathcal{E}\ ,\ \mathsf{let}\ x = \lambda x_1^{\tau}.e_1\ \mathsf{in}\ e_2\ \rangle \Downarrow v}\ \textsc{letRecE} \qquad \frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow \mathsf{true} \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2}{\langle\ \mathcal{E}\ ,\ \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3\ \rangle \Downarrow v_2}\ \textsc{ifTrue}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow \mathsf{false} \qquad \langle\ \mathcal{E}\ ,\ e_3\ \rangle \Downarrow v_3}{\langle\ \mathcal{E}\ ,\ \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3\ \rangle \Downarrow v_3}\ \textsc{ifFalse} \qquad \frac{}{\langle\ \mathcal{E}\ ,\ [\,]^{\tau}\ \rangle \Downarrow [\,]}\ \textsc{nilE}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_1\ \rangle \Downarrow v_1 \qquad \langle\ \mathcal{E}\ ,\ e_2\ \rangle \Downarrow v_2}{\langle\ \mathcal{E}\ ,\ e_1 :: e_2\ \rangle \Downarrow v_1 :: v_2}\ \textsc{consE}$$

$$\frac{\langle\ \mathcal{E}\ ,\ e_0\ \rangle \Downarrow v_0 \qquad \forall j < i.\neg(v_0\ \mathsf{matches}\ p_j) \qquad v_0\ \mathsf{matches}\ p_i \Rightarrow \mathcal{E}_i \qquad \langle\ \mathcal{E} \cup \mathcal{E}_i\ ,\ e_i\ \rangle \Downarrow v}{\langle\ \mathcal{E}\ ,\ \mathsf{match}\ e_0\ \mathsf{with}\ \ \restriction\ p_1 \to e_1 \ldots\ \restriction\ p_k \to e_k\ \rangle \Downarrow v}\ \textsc{matchE}$$

Pattern matching rules:

$$\frac{}{\mathsf{true}\ \mathsf{matches}\ \mathsf{true} \Rightarrow \varnothing}\ \textsc{trueM} \qquad \frac{}{\mathsf{false}\ \mathsf{matches}\ \mathsf{false} \Rightarrow \varnothing}\ \textsc{falseM} \qquad \frac{n \in \mathbb{Z}}{n\ \mathsf{matches}\ n \Rightarrow \varnothing}\ \textsc{intM}$$

$$\frac{}{[\,]\ \mathsf{matches}\ [\,] \Rightarrow \varnothing}\ \textsc{nilM} \qquad \frac{v_1\ \mathsf{matches}\ p_1 \Rightarrow \mathcal{E}_1 \qquad v_2\ \mathsf{matches}\ p_2 \Rightarrow \mathcal{E}_2}{v_1 :: v_2\ \mathsf{matches}\ p_1 :: p_2 \Rightarrow \mathcal{E}_1 \cup \mathcal{E}_2}\ \textsc{consM}$$

$$\frac{}{v\ \mathsf{matches}\ x \Rightarrow \{x \mapsto v\}}\ \textsc{varM} \qquad \frac{}{v\ \mathsf{matches}\ _ \Rightarrow \varnothing}\ \textsc{wildM}$$

As above, note that these rules are incomplete. You need to fill in the implementations of:

```
val matches : Value.t -> Pattern.t -> Env.t option
val eval : Env.t -> Expr.t -> Value.t
```

so that `eval env e` is the result of evaluating e according to the above rules, i.e., it should be v such that $e \Downarrow v$ is derivable (since we're only evaluating well-typed expressions, we know v exists and is unique).

Final Remarks

- Once you're done, you should be able to run

  ```
  dune exec assign9 <filename>
  ```

  to interpret programs in this language. See the `examples` directory for some example programs.

- We're giving a lot of code to you. Take a look at the patterns that appear there. They can help you with the implementation of the parts you need to work on. They can also help guide the code in the final project.

- In particular, make sure to look at the code that appears in each module, e.g., there are useful functions for merging contexts, determining if contexts are disjoint, etc.

- In reality, the code is not terribly interesting; it's a lot of repetition and rule implementation. It's worth thinking about how to simplify it and how to use good abstractions to keep the code clean.

- Happy coding!

Written Problems

You're **not** required to submit anything. That said, we still have a final exam. Here is a list of useful practice problems.

- Progress and Preservation: 8
- Environments: 5, 6, 10, 14, 16, 17, 22
- Pattern Matching: 11, 12, 13, 14, 15